

LAPORAN PRAKTIKUM

Aplikasi Berbasis Platform

PERTEMUAN 9



Disusun oleh:

Christoba joshua

hutagalung

2311102133

S1 IF-11-04

Dosen Pengampu:

Cahyo Prihantoro, S.Kom., M.Eng

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

LINK GIT

https://github.com/Christobacode/Praktikum_ABP.git

Tugas Praktikum Modul 4-5 Flutter

Buat 1 project Flutter yang menampilkan beberapa widget UI berikut:

◆ Yang harus ada:

Container → kotak berwarna

GridView → minimal 6 item (grid)

ListView → 3 item (A, B, C)

ListView.builder → list dari data array

ListView.separated → list + garis pembatas

Stack → tampilan bertumpuk (kotak / text)



Output yang dikumpulkan:

Screenshot hasilnya

Source code

Penjelasan singkat tiap widget

20:30

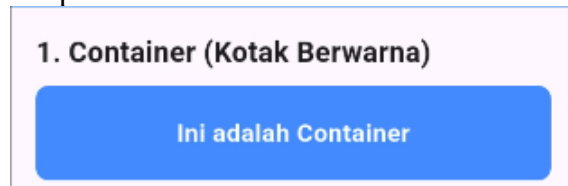
Jawab :

1. Container

Code:

```
// 1. CONTAINER
const SectionTitle(title: '1. Container (Kotak Berwarna)'),
Container(
  height: 60,
  width: double.infinity,
  decoration: BoxDecoration(
    color: Colors.blueAccent,
    borderRadius: BorderRadius.circular(8),
  ),
  alignment: Alignment.center,
  child: const Text(
    'Ini adalah Container',
    style: TextStyle(color: Colors.white, fontSize: 16, fontWeight:
FontWeight.bold),
  ),
),
const SizedBox(height: 24),
```

Output :



Penjelasan:

Pada bagian ini, Container dibuat dengan tinggi 60 dan lebar `double.infinity` agar memenuhi lebar layar. Untuk pewarnaan dan membuat sudut kotak menjadi tumpul, digunakan properti `decoration` memanggil `BoxDecoration` (warna biru dengan radius 8). Properti `alignment`: `Alignment.center` digunakan agar widget `Text` di dalamnya (sebagai `child`) posisinya berada tepat di tengah kotak.

2. GridView

Code:

```
// 3. GRIDVIEW
const SectionTitle(title: '3. GridView (Min. 6 Item)'),
SizedBox(
  height: 200,
  child: GridView.count(
    crossAxisCount: 3,
    mainAxisSpacing: 10,
    crossAxisSpacing: 10,
    children: List.generate(6, (index) => Container(
      color: Colors.purpleAccent,
      alignment: Alignment.center,
      child: Text('Grid ${index + 1}', style: const TextStyle(color: Colors.white)),
    )),
  ),
),
const SizedBox(height: 24),
```

Output:



Penjelasan:

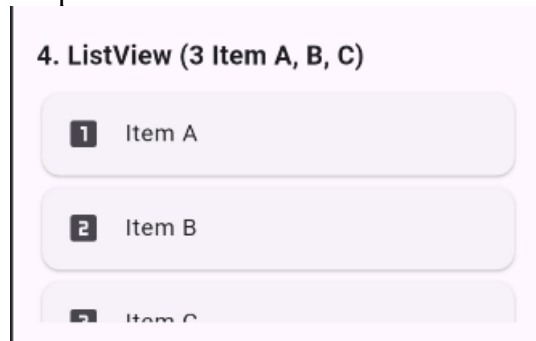
Digunakan `GridView.count` dengan nilai `crossAxisCount: 3` untuk membagi layar menjadi 3 kolom secara horizontal. Properti `mainAxisSpacing` dan `crossAxisSpacing` diberi nilai 10 untuk memberikan jarak (spasi) antar kotak. Untuk memunculkan 6 item secara otomatis tanpa menulis ulang kode, digunakan perulangan `List.generate(6)` yang menghasilkan 6 kotak berwarna ungu berisi teks urutan grid.

3. ListView

Code:

```
// 4. LISTVIEW BIASA
const SectionTitle(title: '4. ListView (3 Item A, B, C)'),
  SizedBox(
    height: 160,
    child: ListView(
      physics: const NeverScrollableScrollPhysics(),
      children: const [
        Card(child: ListTile(leading: Icon(Icons.looks_one), title: Text('Item A'))),
        Card(child: ListTile(leading: Icon(Icons.looks_two), title: Text('Item B'))),
        Card(child: ListTile(leading: Icon(Icons.looks_3), title: Text('Item C'))),
      ],
    ),
  ),
const SizedBox(height: 24),
```

Output:



Penjelasan:

ListView ini dibuat statis dengan langsung mendefinisikan 3 widget Card secara manual didalam children. Hal penting di sini adalah penambahan physics: NeverScrollableScrollPhysics(). Karena ListView ini sudah dibungkus oleh SingleChildScrollView di layar utama, scroll bawaan dari ListView dimatikan agar tidak terjadi bentrok (error saat scroll).

4. ListView.builder

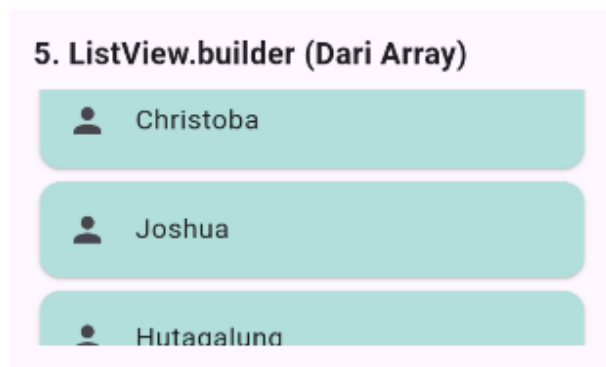
Code:

```
// 5. LISTVIEW.BUILDER
const SectionTitle(title: '5. ListView.builder (Dari Array)'),
SizedBox(
  height: 150,
  child: ListView.builder(
    itemCount: dataList.length,
    itemBuilder: (context, index) {
      return Card(
        color: Colors.teal[100],
        child: ListTile(
          leading: const Icon(Icons.person), // Ikon profil
          title: Text(dataList[index]), // Menampilkan nama
        ),
      );
    },
  ),
),
const SizedBox(height: 24),
```

Array:

```
// Data array nya  
final List<String> dataList = const ['Christoba', 'Joshua', 'Hutagalung'];
```

Output:



Penjelasan:

Tidak seperti ListView biasa, isi dari ListView ini digenerate secara dinamis. Properti itemCount akan menghitung jumlah data dari array dataList (yaitu 3 data: namamu). Kemudian itemBuilder akan melakukan *looping* sebanyak jumlah data tersebut, memanggil elemen array satu per satu menggunakan dataList[index], dan menampilkannya di dalam widget ListTile.

5. ListView.separated

Code:

```
// 6. LISTVIEW.SEPARATED  
const SectionTitle(title: '6. ListView.separated (Garis Pembatas)'),  
  SizedBox(  
    height: 180,  
    child: ListView.separated(  
      itemCount: dataList.length,  
      separatorBuilder: (context, index) => const Divider(color: Colors.red,  
thicknes: 2),  
      itemBuilder: (context, index) {  
        return ListTile(  
          leading: const Icon(Icons.person_outline), // Ikon profil outline  
          title: Text(dataList[index]), // Menampilkan namamu  
        );  
      }  
    ),  
  ),
```

Array:

```
// Data array nya  
final List<String> dataList = const ['Christoba', 'Joshua', 'Hutagalung'];
```

Output:

6. ListView.separated (Garis Pembatas)

 Christoba

 Joshua

 Hutagalung

Penjelasan:

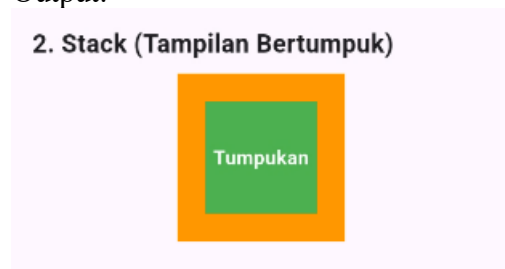
Kodenya bekerja mirip seperti ListView.builder yang mengambil data dari array dataList. Perbedaan utamanya ada pada kewajiban menambahkan properti separatorBuilder. Pada kode ini, *separator* diisi dengan widget Divider berwarna merah dengan ketebalan 2 pixel, yang otomatis akan disisipkan di antara item pertama dan kedua, serta item kedua dan ketiga.

6. Stack

Code:

```
// 2. STACK
const SectionTitle(title: '2. Stack (Tampilan Bertumpuk)'),
Center(
  child: Stack(
    alignment: Alignment.center,
    children: [
      Container(height: 120, width: 120, color: Colors.orange),
      Container(height: 80, width: 80, color: Colors.green),
      const Text('Tumpukan', style: TextStyle(color: Colors.white, fontWeight:
FontWeight.bold)),
    ],
  ),
),
const SizedBox(height: 24),
```


Output:



Penjelasan:

Widget Stack menggunakan properti alignment: Alignment.center agar semua elemen di dalamnya otomatis berada di tengah. Pada array children, urutan penulisan sangat penting. Kotak oranye (ukuran 120x120) dirender paling awal sehingga menjadi latar paling bawah. Di atasnya ditumpuk kotak hijau (80x80), dan di lapisan paling atas ditumpuk widget Text bertuliskan "Tumpukan".